

Hibernate

ddl-auto

Understanding Hibernate Database Schema Generation Strategies

Name: Menna Mohamed El- tayeb

Task: Hibernate ddl-auto

Date: August 2026

1. Introduction

Hibernate is an Object-Relational Mapping (ORM) framework that allows Java applications to communicate with relational databases using Java objects.

When using Spring Boot with Spring Data JPA, the following property controls how Hibernate handles the database schema:

```
spring.jpa.hibernate.ddl-auto=update
```

The `ddl-auto` property determines what Hibernate should do with the database schema when the application starts.

The main values are:

- `validate`
- `update`
- `create`
- `create-drop`
- `none`

These strategies have very different behaviors, especially when it comes to existing tables and data.

2. What Does `ddl-auto` Mean?

`ddl-auto` refers to Hibernate's automatic handling of the database schema.

DDL stands for:

Data Definition Language

DDL commands are SQL commands used to define and modify database structures, such as:

```
CREATE                                TABLE
ALTER                                TABLE
DROP TABLE
```

Hibernate can use the application's Entity classes to determine the expected database structure.

For example, consider this Entity:

```
@Entity
public class Team {

    @Id
    private Long id;

    private String name;

    private int playerCount;

    private boolean active;

    // Getters and Setters
}
```

Hibernate can use this Entity to understand that a corresponding database table should contain columns such as:

```
Team
-----
id
name
playerCount
active
```

The `ddl-auto` value determines what Hibernate should do with this schema.

3. validate

What does it do?

validate tells Hibernate to check whether the existing database schema matches the Entity mappings.

It **does not modify the database schema**.

If the schema matches the Entity mappings, the application can start normally.

If there is a mismatch, Hibernate reports an error during startup.

How does it work?

When the application starts:



For example, if the Entity contains:

```
private String name;  
private int playerCount;
```

Hibernate checks whether the corresponding columns exist in the database.

If a required column is missing or the schema does not match the mapping, Hibernate can fail during startup.

Simple Example

`spring.jpa.hibernate.ddl-auto=validate`

Suppose the database contains:

```
Team
-----
id
name
playerCount
active
```

and the Entity contains the same mapped fields.

Hibernate validates the schema and allows the application to start.

However, if the Entity contains:

```
private String coachName;
```

while the database does not contain the corresponding column, Hibernate does not automatically add it.

Instead, the application may fail during startup because the schema does not match the Entity mapping.

Real-World Case

`validate` is commonly useful in **production environments** when the database schema is managed separately from Hibernate.

For example, a team may use database migration tools such as Flyway or Liquibase to manage schema changes.

In this situation:

Migration		Tool
↓		
Updates	Database	Schema

Hibernate



Validates Schema

Hibernate checks that the application and database agree, but it does not change the production schema automatically.

Important Note

validate is useful when you want Hibernate to act as a **schema checker**, not a schema modifier.

4. update

What does it do?

update tells Hibernate to update the database schema so that it can accommodate the current Entity mappings.

It attempts to modify the existing schema instead of recreating it from scratch.

The goal is to preserve existing data while applying supported schema changes.

How does it work?

Suppose the database initially contains:

Team

id

name

playerCount

active

Then you add this field to the Entity:

```
private String coachName;
```

When the application starts, Hibernate detects the difference between the Entity mapping and the existing schema and may issue a schema change similar to:

```
ALTER TABLE Team ADD coachName VARCHAR(255);
```

The database then becomes:

```
Team
-----
id
name
playerCount
active
coachName
```

Existing records are generally retained.

Simple Example

```
spring.jpa.hibernate.ddl-auto=update
```

Before adding the new field:

```
@Entity
public class Team {

    @Id
    private Long id;

    private String name;

    private int playerCount;

    private boolean active;
}
```

Then we add:

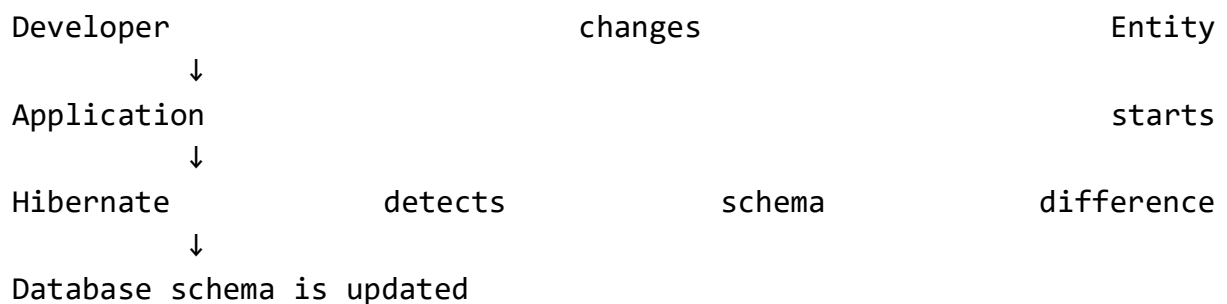
```
private String coachName;
```

Hibernate may update the database schema by adding the corresponding column.

Real-World Case

update is commonly convenient in **development environments** where developers frequently add or modify Entity fields.

For example:



This can save development time because developers do not have to manually recreate the database for every small schema change.

However, relying on update for production databases is generally not recommended. Production systems usually use controlled database migration tools such as Flyway or Liquibase.

Important Note

update does not mean:

"Hibernate will make every possible database change perfectly and safely."

Schema changes can have limitations and potentially dangerous consequences, especially in complex production systems.

For that reason, controlled migrations are preferred in serious production environments.

5. create

What does it do?

create tells Hibernate to create the database schema based on the current Entity mappings when the application starts.

Existing database objects managed by the generated schema can be removed and recreated.

Therefore, existing data can be lost.

How does it work?

When the application starts:



Hibernate builds the schema from the current Entity definitions.

Simple Example

```
spring.jpa.hibernate.ddl-auto=create
```

Suppose the Entity is:

```
@Entity
public class Team {

    @Id
    private Long id;

    private String name;
```

```
        private                int                playerCount;  
    }
```

Hibernate creates a table based on this mapping.

If the database already contains data in a schema that Hibernate recreates, that data may be lost.

Real-World Case

`create` can be useful in certain **development or testing environments** where the database is disposable.

For example, a developer may want to start with a fresh database schema every time the application starts.

It can also be useful for temporary environments where preserving database data is not important.

Important Note

Do not use `create` casually on a database containing important data.

It is designed for situations where recreating the schema is acceptable.

6. create-drop

What does it do?

`create-drop` tells Hibernate to:

1. Create the database schema when the application starts.
2. Drop the schema when the Hibernate SessionFactory is closed.

In a Spring Boot application, this generally means the schema is created for the application's runtime and removed when the application shuts down.

How does it work?

The lifecycle is:



Simple Example

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Suppose we have:

```
@Entity
public class Team {

    @Id
    private Long id;

    private String name;

    private int playerCount;
}
```

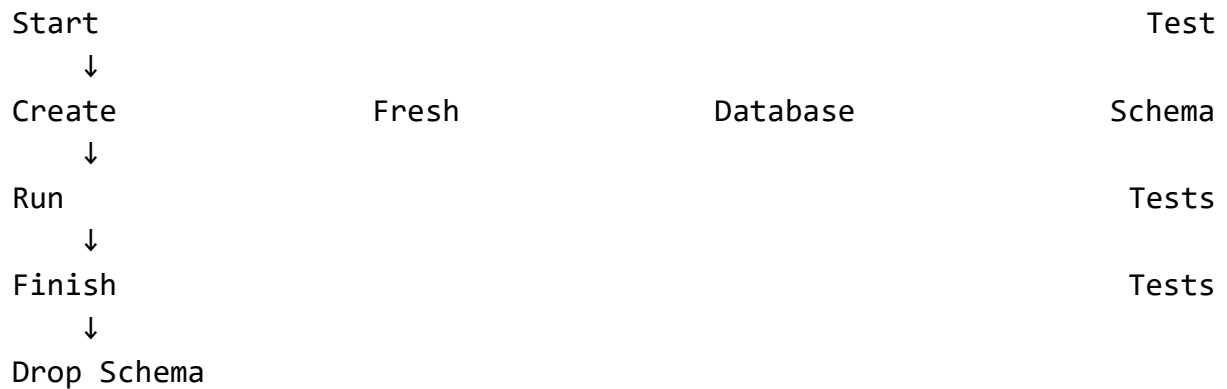
When the application starts, Hibernate creates the Team table.

When the application shuts down and the Hibernate context is closed, Hibernate drops the schema it manages.

Real-World Case

create-drop is particularly useful for **automated testing**.

For example, when running integration tests:



This gives each test environment a clean database state and prevents test data from remaining after the test execution.

It is also useful for temporary development environments where the database does not need to persist between application runs.

Important Note

create-drop should not be used with important production data because the schema is intended to be removed when the Hibernate context is closed.

7. none

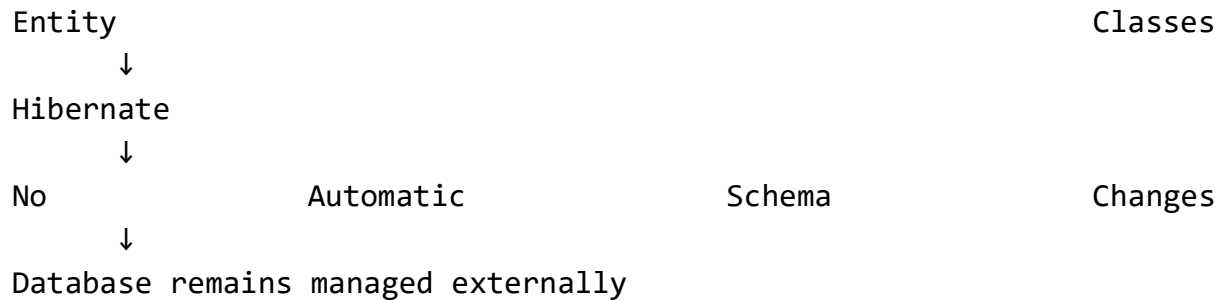
What does it do?

none tells Hibernate not to automatically create, update, validate, or drop the database schema.

Hibernate does not manage the database schema through ddl-auto.

How does it work?

The process is simply:



Hibernate still uses the Entity mappings to perform ORM operations such as queries, inserts, updates, and deletes.

The important point is that Hibernate does not automatically modify the database schema.

Simple Example

```
spring.jpa.hibernate.ddl-auto=none
```

Suppose the database already has:

```
Team
-----
id
name
playerCount
active
```

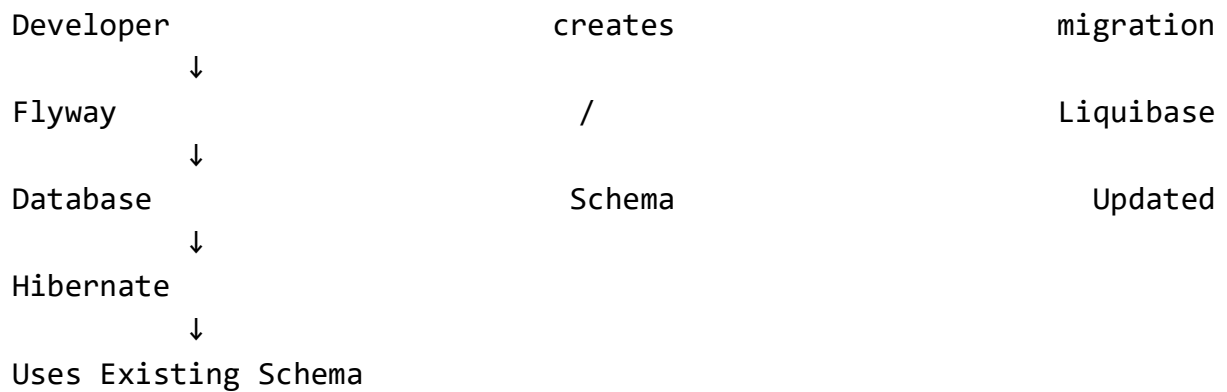
Hibernate will not create or modify this table using the `ddl-auto` mechanism.

The database schema must already be prepared by another process.

Real-World Case

none can be useful in **production environments** where database schema changes are fully managed by a separate database migration process.

For example:



This gives the team explicit control over database changes.

Important Note

none does not mean Hibernate stops working.

It only means Hibernate does not automatically manage the database schema through this property.

Hibernate can still perform normal ORM operations.

8. Comparison Table

Value	What does it do?	Real-World Case
validate	Checks whether the existing database schema matches the Entity mappings without changing the schema.	Production environments where migrations are managed separately and Hibernate should verify the schema.
update	Attempts to update the existing schema to accommodate Entity changes while preserving existing data where possible.	Development environments where Entity classes change frequently.

create	Recreates the database schema based on the Entity mappings when the application starts. Existing data may be lost.	Disposable development or testing databases where a fresh schema is needed.
create-drop	Creates the schema at startup and drops it when the Hibernate context is closed.	Automated tests and temporary environments.
none	Does not automatically manage the database schema.	Production environments where schema management is handled completely by migration tools or DBAs.

9. Side-by-Side Behavior

A simple way to remember the five values is:

Value	Checks Schema	Modifies Schema	Creates Schema	Drops Schema
validate	✓	✗	✗	✗
update	✓	✓	Can create missing structures	✗ intentionally
create	✗ as its main purpose	Recreates	✓	Can drop existing generated schema
create-drop	✗ as its main purpose	Recreates	✓	✓
none	✗	✗	✗	✗

Note: Exact DDL behavior can vary by Hibernate version, database dialect, and schema state. The table describes the intended strategy at a high level.

10. Real-World Recommendation

There is no single `ddl-auto` value that is best for every environment.

A common approach is:

Development

```
spring.jpa.hibernate.ddl-auto=update
```

This can make development easier when the Entity model is changing frequently.

Automated Tests

```
spring.jpa.hibernate.ddl-auto=create-drop
```

This can provide a fresh schema for temporary test runs.

Production

Production applications commonly avoid automatic schema recreation.

A typical approach is to use:

Flyway	/	Liquibase
+		
Hibernate		

For example:

```
spring.jpa.hibernate.ddl-auto=validate
```

or:

```
spring.jpa.hibernate.ddl-auto=none
```

The exact choice depends on how the organization manages database schemas.

11. Common Mistakes

Mistake 1: Thinking update is a backup strategy

update does not create a backup of the database.

It only controls Hibernate's schema update behavior.

Mistake 2: Using create with important data

Using create can recreate the schema and result in data loss.

It should only be used when recreating the database schema is acceptable.

Mistake 3: Using create-drop on production data

create-drop is designed for temporary environments.

The schema is dropped when the Hibernate context is closed.

Mistake 4: Thinking none disables Hibernate

It does not.

Hibernate can still perform:

INSERT
SELECT
UPDATE
DELETE

It simply does not automatically manage the database schema through ddl-auto.

Mistake 5: Assuming validate fixes the database

validate only checks the schema.

If there is a mismatch, it reports a problem rather than automatically fixing it.

12. Interview Cheat Sheet

validate

Think:

"Check, don't change."

Hibernate checks the schema against the Entity mappings.

update

Think:

"Adapt the existing schema."

Hibernate attempts to update the schema to accommodate Entity changes.

Best remembered as a convenient development option.

create

Think:

"Fresh schema on startup."

Hibernate recreates the schema based on the Entity mappings.

Existing data may be lost.

create-drop

Think:

"Create → Use → Drop."

The schema exists during the application's runtime and is dropped when the Hibernate context closes.

Great for temporary test environments.

none

Think:

"Hibernate doesn't manage the schema."

The database schema is managed externally.

13. Final Summary

The five `ddl-auto` values represent different levels of Hibernate's responsibility for managing the database schema:

validate

↓

Check only

update

↓

Update existing schema

create

↓

Recreate schema at startup

create-drop

↓

Create at startup
Drop at shutdown

none

↓

Do nothing to schema

The most important concept is to understand the difference between **schema management** and **normal database operations**.

ddl-auto controls how Hibernate handles the **database structure**. It does not determine whether Hibernate can perform normal operations such as inserting, updating, deleting, or retrieving records.

For real-world production systems, database schema changes are often managed explicitly using migration tools such as **Flyway** or **Liquibase**, rather than relying on automatic Hibernate schema updates.

14. One-Line Revision Table

Value	Easy Way to Remember
validate	Check only
update	Update existing schema
create	Recreate at startup
create-drop	Create at startup, drop at shutdown

Final Rule to Remember

Development → update

Testing → create-drop

Production → Usually controlled migrations + validate or n